

Attention Models

Capturing data dependencies

Goal of Self-Attention

Compute a **context vector** for each input element that integrates information from all other input elements.

Example:

- for word-2 in a sentence
- Context vector $\mathbf{z}^{(2)}$ is computed.
- contribution of each input element to $\mathbf{z}^{(2)}$ is determined by attention weights α_{21} to α_{2T} .

Representing the Words

- Each word in the sentence is turned into a small numeric representation (an embedding). Think of it like assigning a unique "fingerprint" to each word that captures its meaning.
- For example:
 - "My" → [small list of numbers, say [0.2, 0.1, 0.3]]
 - "learning" → [another list of numbers, say [0.4, 0.2, 0.1]]
 - And so on for the other words.

Focus on What's Important



The attention mechanism asks, "**Which words matter most for the task we're working on?**"



For example:

If the task is understanding the topic of the sentence, "learning" and "lesson" might be more **important** than "my" or "with."

Attention assigns each word a "**score**" based on how much **focus** it should get.

Words like "learning" and "lesson" might get higher scores than others.

Example with the
Sentence: "My
learning starts
with this lesson"

For "**My**", its context vector combines some information from "learning," "starts," and so on.

For "**learning**", its context vector gives more focus to "**lesson**" and maybe "**starts**".

For "**lesson**", its context vector might emphasize "**learning**" and "**this.**"

so, context
vector is for
every token?

Every token gets its own focus

- Each token (word or subword) **attends** to all other tokens in the sequence. The attention mechanism calculates scores for how much each token should "**focus**" on the others.

Create a unique context vector per token

- Using the attention scores and embeddings of all tokens, a unique context vector is created for each token. This context vector **summarizes** the information that token "sees" from the rest of the sequence.

input text : "My learning starts with this lesson"

Tokens →

My

learning

starts

...

lesson

$x^{(1)}$

$x^{(2)}$

$x^{(3)}$

$x^{(T)}$

Embeddings →

.4 .1 .8

.5 .8 .6

.5 .8 .6

.0 .8 .5

Attention weights →

α^{21}

α^{22}

α^{23}

α^{2T}

Context Vector →

.4 .6 .5

z^2

.4 .6 .5

The context vector $z^{(2)}$ is computed as a combination of all input vectors **weighted** with respect to input element $x^{(2)}$

input text : "My learning starts with this lesson"

Tokens →

My

learning

starts

...

lesson

$x^{(1)}$

$x^{(2)}$

$x^{(3)}$

$x^{(T)}$

Embeddings →

.4 .1 .8

.5 .8 .6

.5 .8 .6

.0 .8 .5

Attention weights →

α^{21}

α^{22}

α^{23}

α^{2T}

Context Vector →

.4 .6 .5

z^2

The context vector $z^{(2)}$ is computed as a combination of all input vectors **weighted** with respect to input element $x^{(2)}$

Attention score

- Before we compute “attention weights” we compute the “attention scores”
- For example, in the sentence "My learning starts with this lesson":
 - "My" → embedding vector for the word "My"
 - "learning" → embedding vector for the word "learning"
 - "starts" → embedding vector for "starts"
 - And so on...

Compute the Attention Scores (Relevance)



Raw attention score between two tokens (say "**learning**" and "**starts**") is computed by measuring the similarity of their embeddings.

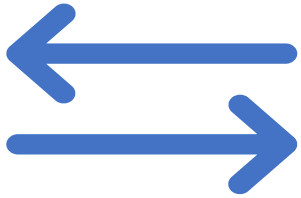


A common way to do this is through the **dot product** of their embedding vectors.

If two embeddings are **similar** (i.e., their meanings are related), the **dot product will be higher**.

If they are very different (i.e., unrelated), the dot product will be lower.

So, for each pair of tokens, we compute...



Raw Attention Score(i, j) =
Embedding of Token _{i} ·
Embedding of Token _{j}

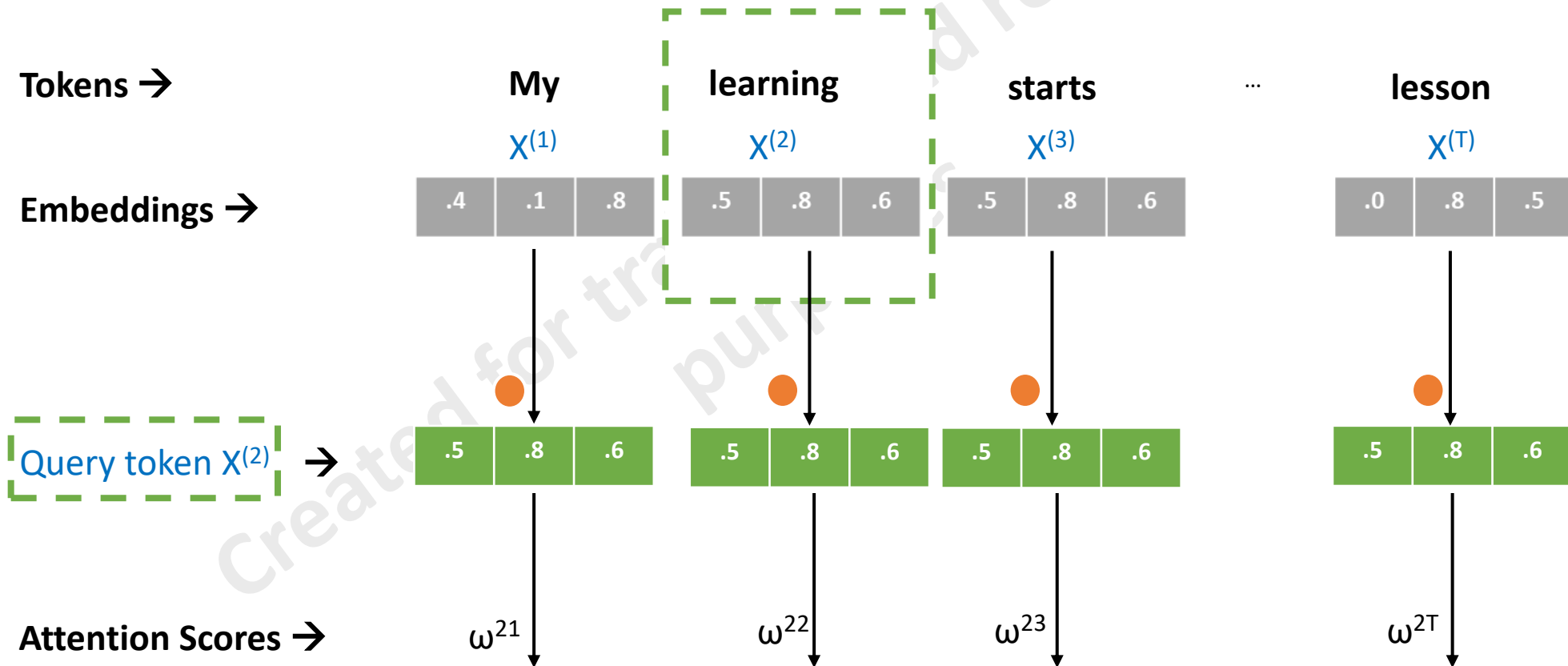


For example:

The attention score between "**learning**" and "**starts**" will reflect how similar those two words are in meaning.

The score between "**learning**" and "**my**" will likely be lower, since "my" is a less relevant word in understanding "**learning**".

Attention score - illustration



Dot Product of Query and Tokens

- Query embedding for Token 2: $Q2 = [1, 2, 3]$
- Embedding for Token 1 : $X1 = [4, 5, 6]$
- attention score between Token 2 and Token 1 is computed as
 - $\text{score}(2, 1) = Q2 \cdot X1 = (1 \times 4) + (2 \times 5) + (3 \times 6) = 32$
- The attention scores for the sentence will look like
 - `tensor([0.9544, 1.4950, 1.4754, 0.8434, 0.7070, 1.0865])`



Attention scores looks like ...

	Learning	starts	with	this	lesson	my
Learning	0	1.6351	0.0849	1.402	1.9116	1.2037
starts	1.6351	0	1.8693	0.655	0.8913	1.7262
with	1.2236	1.3898	0	0.8938	1.331	1.1064
this	0.782	1.2392	1.6986	0	1.6156	0.9815
lesson	1.9116	1.4218	1.2965	0.5833	0	1.4052
my	1.7886	0.0592	0.8523	0.3756	1.0399	0

Attention scores – some key points

can have a wide range of values

- can cause certain tokens to dominate the attention mechanism
- causing the model to focus too much on a small subset of tokens and ignoring others
- might lead to unintended biases in the attention mechanism
- large values can lead to numerical instability during training

not immediately interpretable as probabilities or weights

So we need to normalize the attention scores

Summary of Why Normalization is Needed

Interpretability

Converts raw scores into probabilities, making them easier to understand and use.

Prevents dominance

Avoids the issue where some tokens' scores are too large and overshadow others.

Numerical stability

Ensures that calculations remain stable and manageable during training.

Focus control

Helps the model focus more on important tokens and less on irrelevant ones.

Efficiency

Facilitates parallel computations across multiple tokens, making it easier to train large models.

Attention weight - illustration

Tokens →

Embeddings →

Query token $X^{(2)}$ →

Attention Scores →

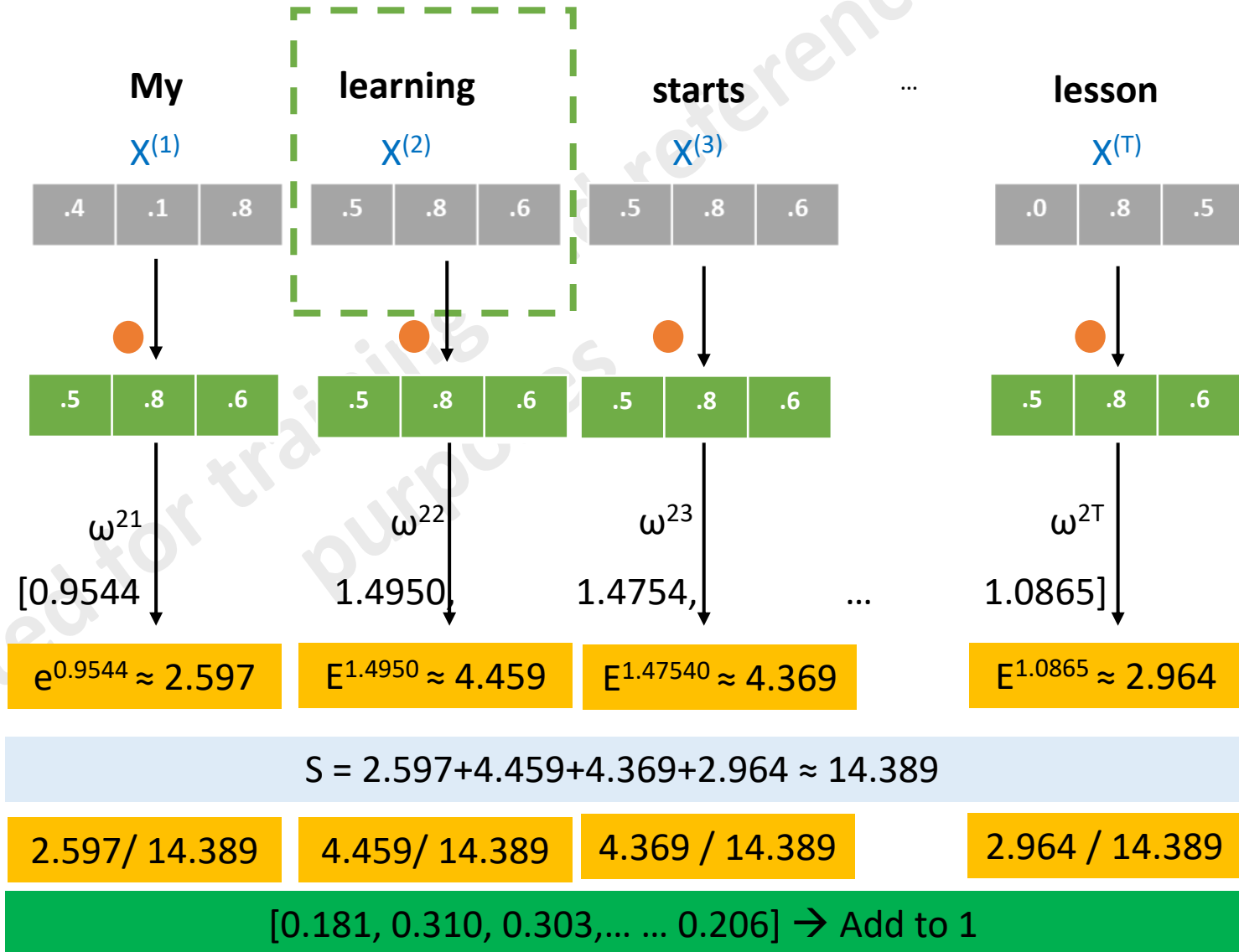
$v \rightarrow$

Exponentiate Each Value

Sum of Exponentials

Softmax for Each Value

softmax(v) –
Attention weight



```
tensor([ [0.2098, 0.2006, 0.1981, 0.1242, 0.1220, 0.1452],  
        [0.1385, 0.2379, 0.2333, 0.1240, 0.1082, 0.1581],  
        [0.1390, 0.2369, 0.2326, 0.1242, 0.1108, 0.1565],  
        [0.1435, 0.2074, 0.2046, 0.1462, 0.1263, 0.1720],  
        [0.1526, 0.1958, 0.1975, 0.1367, 0.1879, 0.1295],  
        [0.1385, 0.2184, 0.2128, 0.1420, 0.0988, 0.1896] ])
```

attention weight
tensor

Normalize each row so that
the values in each row sum to
1

Context Vector



The concept of an **enhanced context vector**, $z^{(2)}$, refers to an embedding or representation that encapsulates information about both the specific element $x^{(2)}$ and its relationship with all other input elements $x^{(1)}, x^{(3)}, \dots, x^{(T)}$ in a sequence.

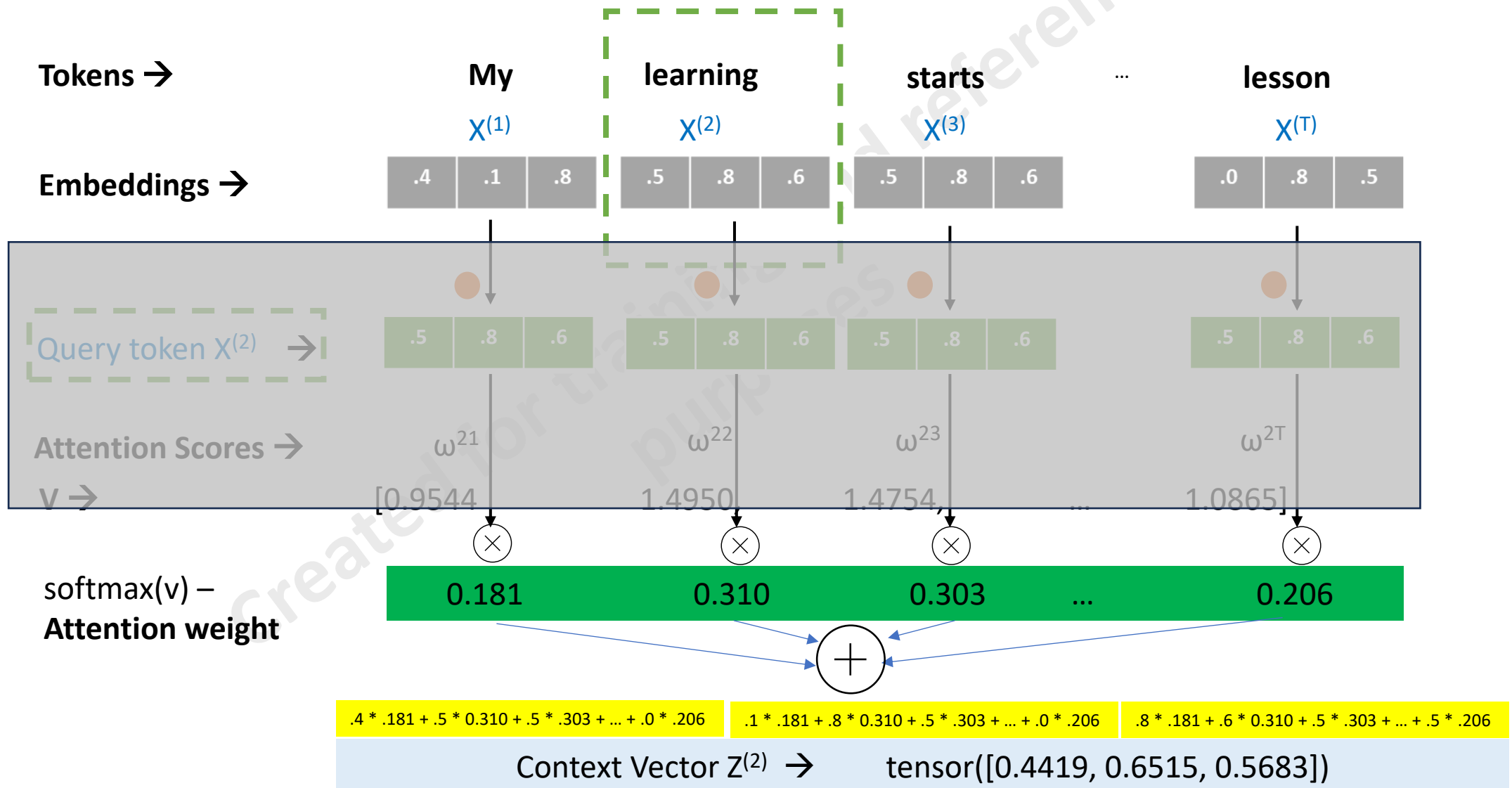


a weighted combination of input elements where the weights reflect the relevance of each input element to $x^{(2)}$.



In the self-attention mechanism, for each input $x^{(i)}$, we calculate attention weights $\alpha^{(i)}$ that measure how much $x^{(i)}$ attends to every other input element in the sequence.

How the context vectors is computed



Understanding context vector



Define the Input : We have 6 tokens, each represented by a 3-dimensional embedding

- $X = [x_1, x_2, x_3, x_4, x_5, x_6]$

Example embeddings
(random values for illustration)

- $x_1 = [1.2, 0.7, -0.3],$
- $x_2 = [0.5, 1.3, -1.1],$
- $x_4 = [0.3, 0.5, -0.7],$
- $x_5 = [-0.6, 1.2, 0.8],$
- $x_6 = [1.0, -0.2, 0.1]$

Define Attention Weights

- Random attention weights for **Token 2** with respect to all tokens (including itself):
 - $A_{21} = 0.1, \alpha_{22} = 0.4, \alpha_{23} = 0.2, \alpha_{24} = 0.1, \alpha_{25} = 0.15, \alpha_{26} = 0.05$
- Note: Attention weights sum to 1:

$$\sum_{j=1}^6 \alpha_{2j} = 0.1 + 0.4 + 0.2 + 0.1 + 0.15 + 0.05 = 1$$

Compute the Context Vector

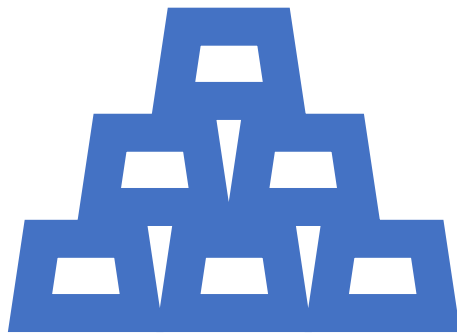
- Substitute the embeddings and weights
 - $Z_2 = (0.1 \cdot x_1) + (0.4 \cdot x_2) + (0.2 \cdot x_3) + (0.1 \cdot x_4) + (0.15 \cdot x_5) + (0.05 \cdot x_6)$
- Perform the computation for each dimension
 - $Z_2 = 0.1 [1.2, 0.7, -0.3] +$
 $0.4 [0.5, 1.3, -1.1] +$
 $0.2 [1.1, -0.8, 0.4] +$
 $0.1 [0.3, 0.5, -0.7] +$
 $0.15[-0.6, 1.2, 0.8] +$
 $0.05[1.0, -0.2, 0.1]$
- The **context vector z_2** for Token 2 is
 - $Z_2 = [0.53, 0.65, -0.435]$

all_context_vecs

- In the resulting output tensor, each row contains a 3-dimensional context vector

```
tensor( [[0.4421, 0.5931, 0.5790],  
         [0.4419, 0.6515, 0.5683],  
         [0.4431, 0.6496, 0.5671],  
         [0.4304, 0.6298, 0.5510],  
         [0.4671, 0.5910, 0.5266],  
         [0.4177, 0.6503, 0.5645]])
```


Intuition Behind the Context Vector



- The **context vector $\mathbf{z}_2$** captures a weighted combination of the contributions from all tokens, including Token 2 itself.
- **Self-Importance (Token 2):**
 - The weight $\alpha_{22}=0.4$ is the largest, meaning that Token 2's own embedding contributes the most to its context vector.
 - This reflects the self-attention mechanism emphasizing the token itself.
- **Contribution from Other Tokens:**
 - Each of the other tokens contributes proportionally to its attention weight.
 - Token 3 ($\alpha_{23} = 0.2$) has the second-highest contribution, indicating a moderate relationship with Token 2.
 - Token 6 ($\alpha_{26} = 0.05$) contributes the least, reflecting its weak relevance to Token 2.

Here's how this works



Each token's embedding is projected onto the **same 3-dimensional space**.



The attention weights (α_{2j}) scale each token's contribution in that space.



The final vector z_2 is a **summary** that reflects all token influences, weighted by their importance.

Each
dimension of
the context
vector serves
as a shared
"feature axis".

- These axes could represent abstract linguistic or semantic properties (e.g., **topic**, **sentiment**, or **syntax**).
- When embeddings for 6 tokens are combined, they are mixed in such a way that the resulting context vector encapsulates these properties.
- **Example:** Suppose the dimensions represent:
 1. **Sentiment Strength:** Positive vs. Negative.
 2. **Relevance to Query:** High vs. Low.
 3. **Syntactic Role:** Subject vs. Object.
- Each token embedding has values for these dimensions, and the attention weights ensure that:
- Highly relevant tokens dominate these dimensions.
- Irrelevant tokens contribute minimally, reducing noise.

Wondering !!

- Sentence : “My learning starts with this lesson”
- For the token “My”
 - “learning” and/or “lesson” may have more influence
- How is this ensured?

$$Z = \begin{bmatrix} \mathbf{z}_1 \\ \mathbf{z}_2 \\ \vdots \\ \mathbf{z}_T \end{bmatrix} \rightarrow Z = \begin{bmatrix} z_{1,1} & z_{1,2} & z_{1,3} \\ z_{2,1} & z_{2,2} & z_{2,3} \\ z_{3,1} & z_{3,2} & z_{3,3} \\ z_{4,1} & z_{4,2} & z_{4,3} \\ z_{5,1} & z_{5,2} & z_{5,3} \\ z_{6,1} & z_{6,2} & z_{6,3} \end{bmatrix} \rightarrow \text{tensor}([[0.4421, 0.5931, 0.5790], [0.4419, 0.6515, 0.5683], [0.4431, 0.6496, 0.5671], [0.4304, 0.6298, 0.5510], [0.4671, 0.5910, 0.5266], [0.4177, 0.6503, 0.5645]])$$

Sentence-Level Context Representation

- **Matrix Form:** If we compute the context vector for all T tokens, we obtain a **matrix of context vectors** Z , Each row z_i is the context vector for token i .

Use in Transformers

1

this context matrix Z serves as the output of the self-attention layer.

2

passed to subsequent layers (e.g., feed-forward layers, additional attention layers, etc.)

3

to further refine the representations for downstream tasks like translation, classification, or summarization.



Why "Lesson" Gets a Higher Weight to "Learning"

- **Embedding Training Encodes Semantic Relationships**
 - The closeness in the embedding space is based on **patterns of co-occurrence** in the training data.
 - For example:
 - If "lesson" and "learning" often appear together in sentences (e.g., "I learned a lesson"), their embeddings will be adjusted during training to reflect this relationship.

Attention Model Uses Learned Patterns

Training on Massive Data

- Learns to optimize attention weights so the output matches the expected result.

Optimization Objective

The training process uses a **loss function**, such as cross-entropy, to compare the model's predictions with the true output.

Assign higher attention weights to words that contribute more to the prediction.

Minimize attention to irrelevant or unrelated words.

Context Vectors Are Computed Per Batch

Training involves batches of sentences (or sequences)

A batch contains multiple sentences or sequences, often padded to the same length for efficient computation.

The model processes the entire batch at once, using **parallel computation**.

Context vectors are computed for all tokens in the batch

For each token in each sentence, the model computes its context vector by considering all other tokens in the same sentence (or sequence) as input.

This happens in parallel for all sentences in the batch.

How It Works in Practice

Input Representation

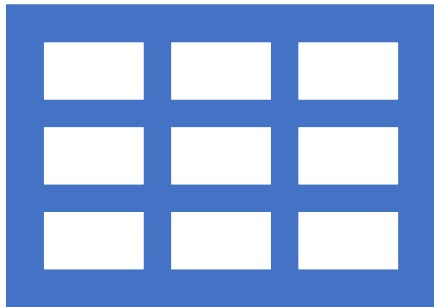
A batch of tokenized sentences is fed into the Transformer model.

Example batch (with padding)

Sentence 1: ["I", "love", "AI", ".", "<PAD>", "<PAD>"]

Sentence 2: ["Transformers", "are", "great", ".", "<PAD>", "<PAD>"]]

Per sentence - attention



Self-Attention Within Each Sentence

The attention mechanism computes relationships between tokens **within each sentence**.

For "Sentence 1," the context vector for "love" is computed by attending to "I," "love," "AI," ".", and any padding tokens.

Parallel Processing Across Sentences

This process happens simultaneously for all sentences in the batch, but the attention mechanism doesn't mix tokens across sentences.

Padding tokens are ignored (or given minimal weight) in the attention mechanism.

implement the self- attention mechanism

using the three trainable weight matrices W_q , W_k and W_v

Let $x = \{x_{(1)}, x_{(2)}, \dots, x_{(T)}\}$ represent the input tokens, where each token is represented as an embedding vector.

These embeddings are typically pre-trained embeddings or random initialization.

Recap of the Setup

Suppose the size of the token embedding is
 $d_{\text{emb}} = 6$
(i.e., each token is represented by a
6-dimensional vector).

The size of the **query**, **key**, and **value** vectors is
 $d_{\text{attn}} = 3$ (i.e., after transformation, we want
these vectors to be 3-dimensional).

matrices

Define the Input Embedding $x_{(i)}$

- $x_{(i)} = [0.1, .2, .3, .4, .5, .6]^T$

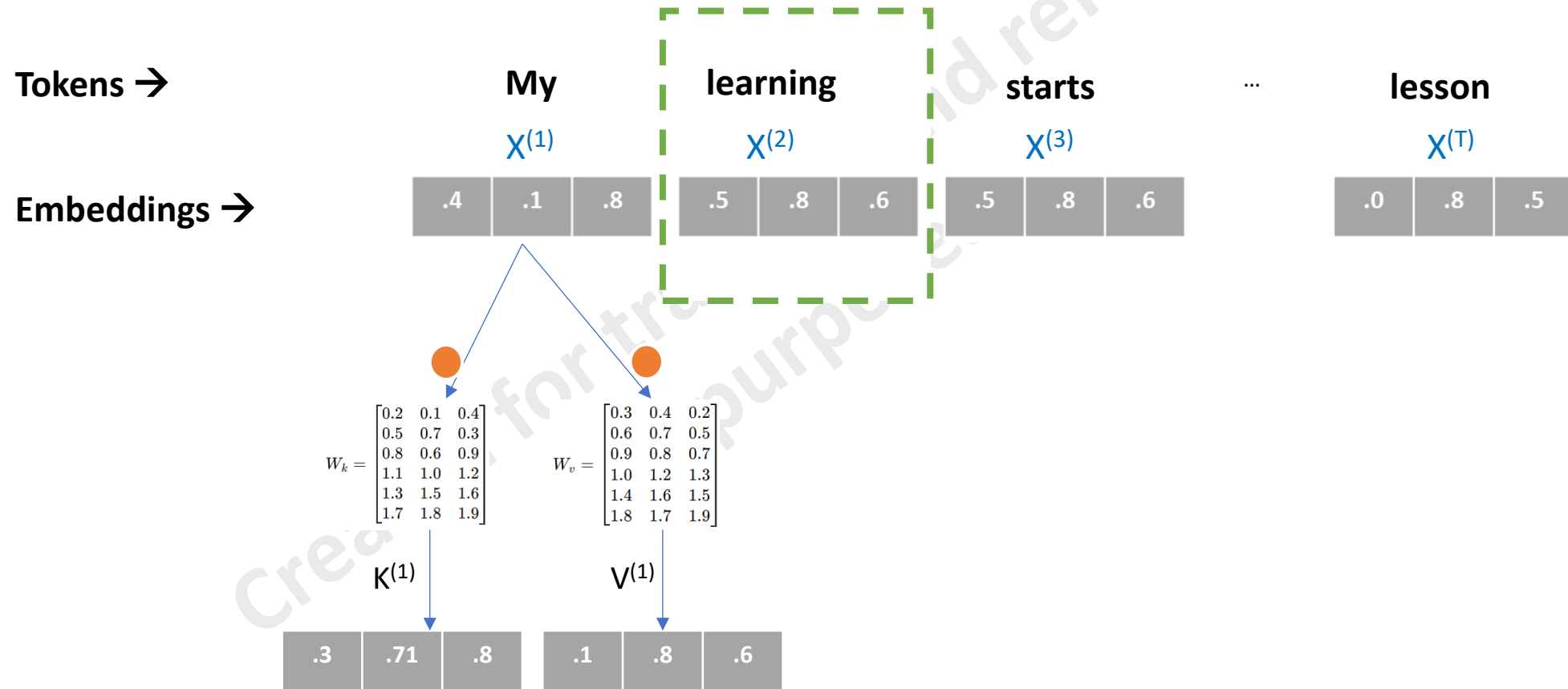
Define the Weight Matrices W_q , W_k , and W_v

- are trainable parameters
- W_q (size 6×3): Transform input into query vectors.
- W_k (size 6×3): Transform input into key vectors.
- W_v (size 6×3): Transform input into value vectors.

example weight matrices

Query weight matrix W_q (size 6×3)	Key weight matrix W_k (size 6×3)	Value weight matrix W_v (size 6×3)
$W_q = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \\ 1.6 & 1.7 & 1.8 \end{bmatrix}$	$W_k = \begin{bmatrix} 0.2 & 0.1 & 0.4 \\ 0.5 & 0.7 & 0.3 \\ 0.8 & 0.6 & 0.9 \\ 1.1 & 1.0 & 1.2 \\ 1.3 & 1.5 & 1.6 \\ 1.7 & 1.8 & 1.9 \end{bmatrix}$	$W_v = \begin{bmatrix} 0.3 & 0.4 & 0.2 \\ 0.6 & 0.7 & 0.5 \\ 0.9 & 0.8 & 0.7 \\ 1.0 & 1.2 & 1.3 \\ 1.4 & 1.6 & 1.5 \\ 1.8 & 1.7 & 1.9 \end{bmatrix}$

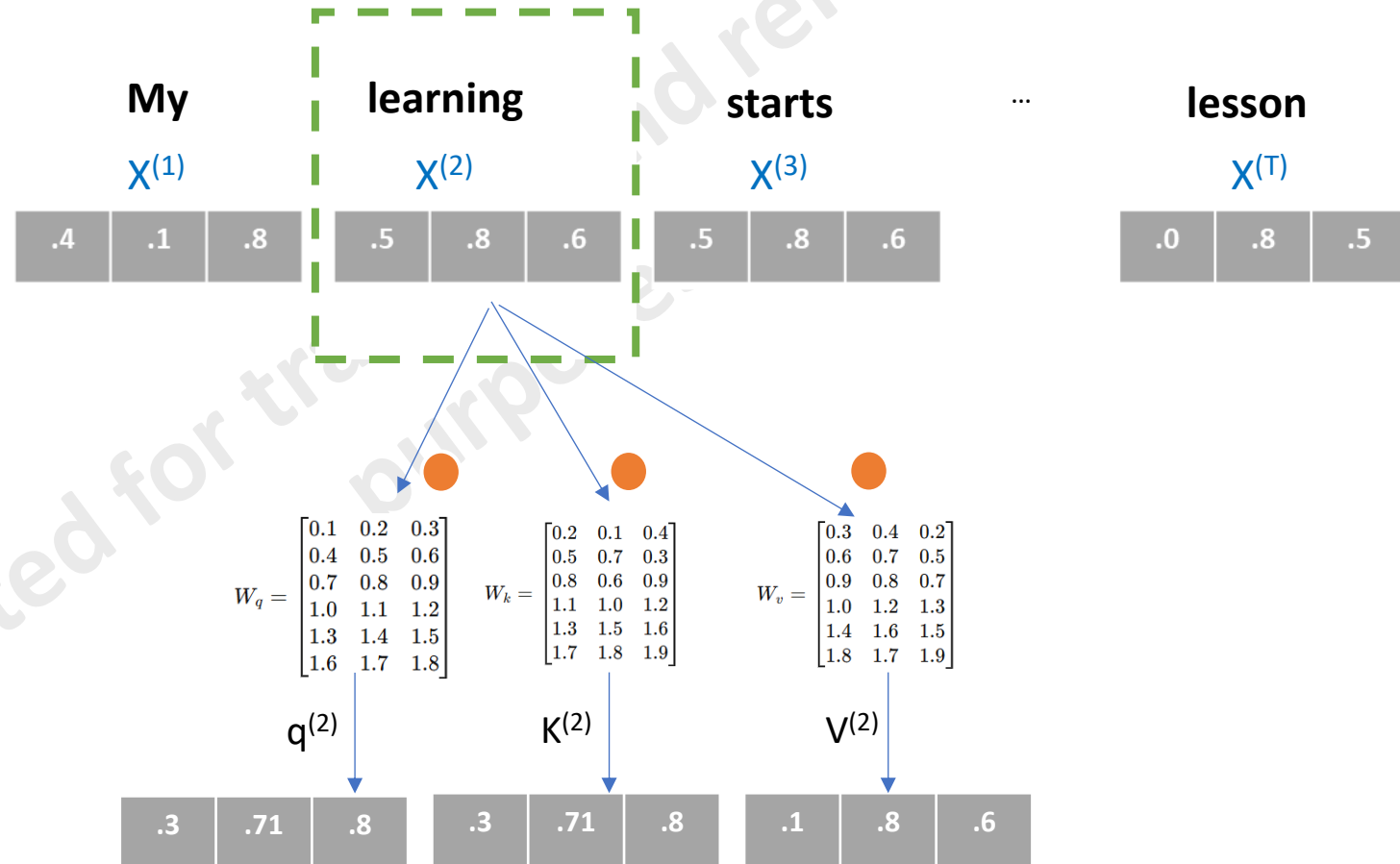
three trainable weight matrices & vectors



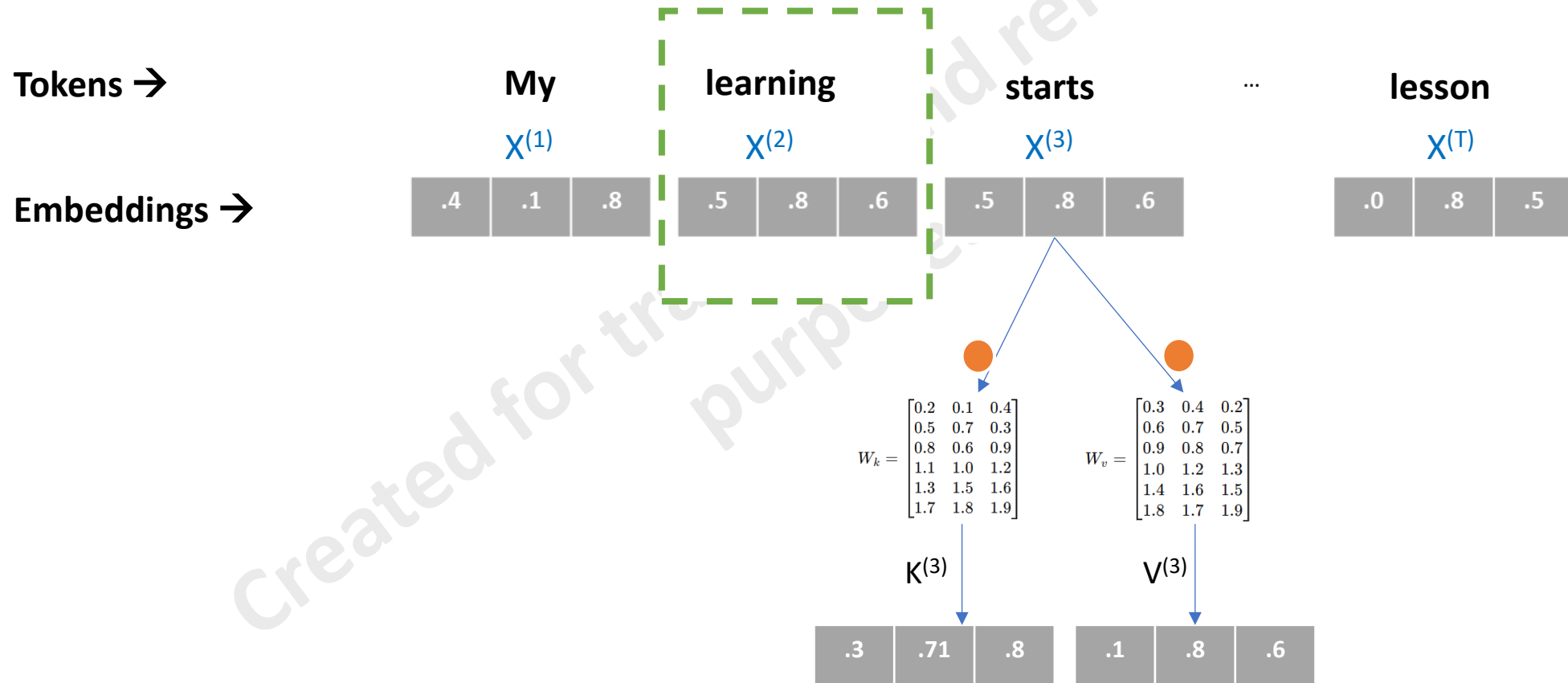
three trainable weight matrices & vectors

Tokens →

Embeddings →



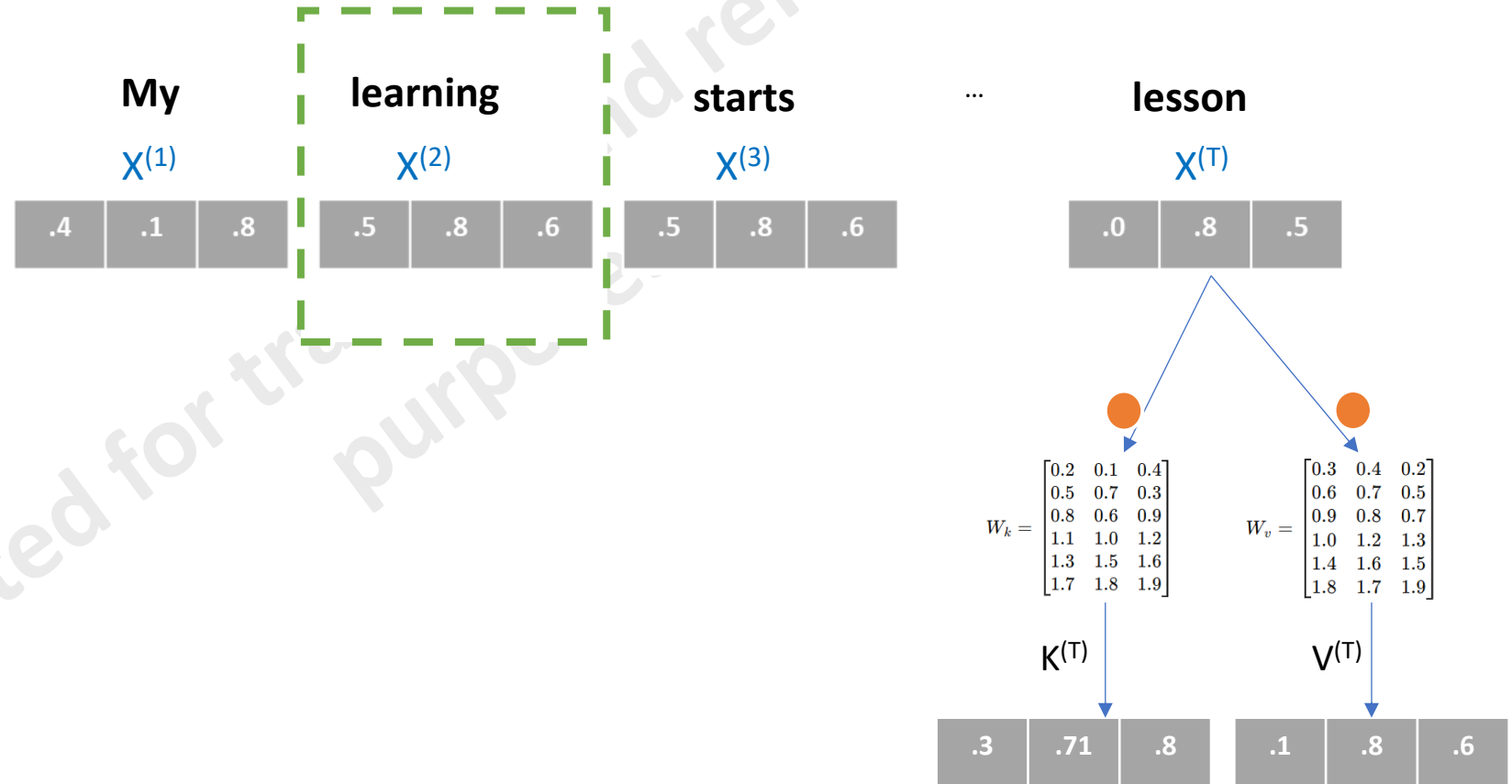
three trainable weight matrices & vectors



three trainable weight matrices & vectors

Tokens →

Embeddings →



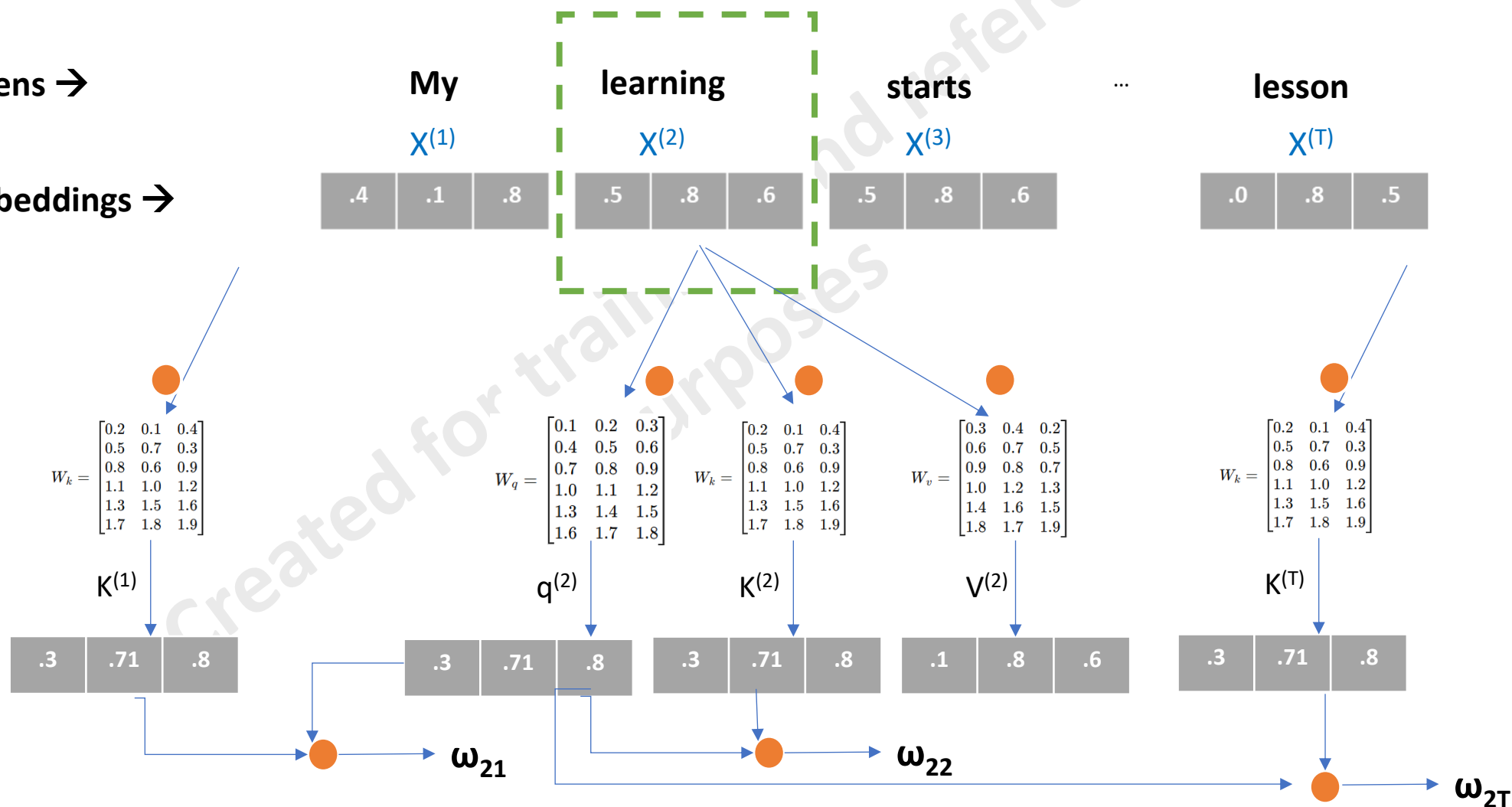
Weight matrices - dimensions

- **Dimensions of the Weight Matrices** (W_q, W_k, W_v)
- **Shape:** $d_{\text{emb}} \times d_{\text{attnd}}$
 - d_{emb} : The embedding size of each token.
 - d_{attnd} : The size of the output query, key, or value vectors.
- These weight matrices are **shared across all tokens**, meaning they are applied to every token embedding in the sequence.

Computing attention score

Tokens →

Embeddings →



Summary so far ...

Query vector

$$(q_i) = x_i \cdot W_Q$$

W_Q transforms the token embedding into a vector of dimension d_k .

Key vector

$$(k_i) = x_i \cdot W_K$$

W_K transforms the token embedding into a vector of dimension d_k .

Value vector

$$(v_i) = x_i \cdot W_V$$

W_V transforms the token embedding into a vector of dimension d_k .

In Matrix Form

Query Matrix

$$(Q) = X \cdot W_Q$$

W_Q transforms the token embedding into a vector of dimension d_k .

Key vector

$$(K) = X \cdot W_K$$

W_K transforms the token embedding into a vector of dimension d_k .

Value vector

$$(V) = X \cdot W_V$$

W_V transforms the token embedding into a vector of dimension d_k .

Q, K and V matrices

My	.4	.1	.8
learning	.5	.8	.6
starts	.5	.8	.6
...			
lesson	.0	.8	.5

$$W_q = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \\ 1.6 & 1.7 & 1.8 \end{bmatrix}$$

$$W_k = \begin{bmatrix} 0.2 & 0.1 & 0.4 \\ 0.5 & 0.7 & 0.3 \\ 0.8 & 0.6 & 0.9 \\ 1.1 & 1.0 & 1.2 \\ 1.3 & 1.5 & 1.6 \\ 1.7 & 1.8 & 1.9 \end{bmatrix}$$

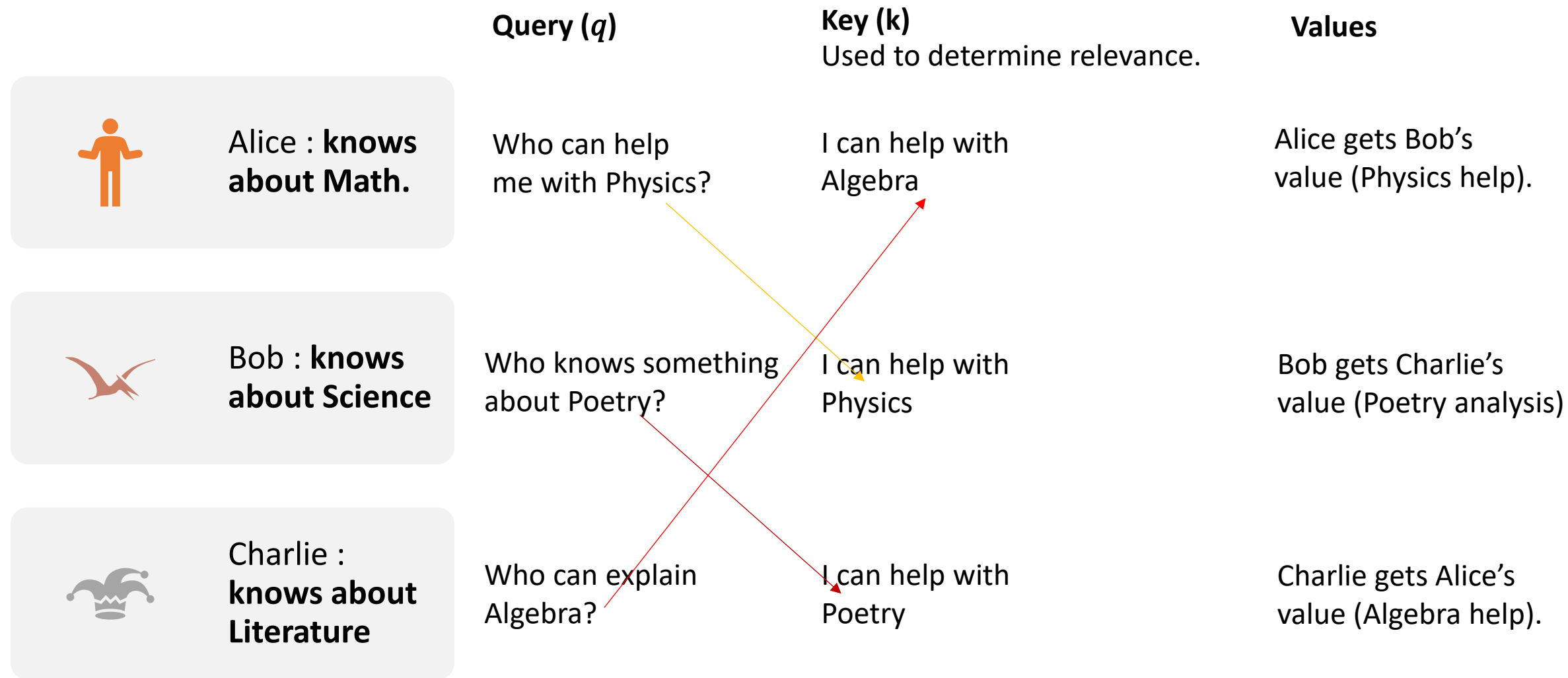
$$W_v = \begin{bmatrix} 0.3 & 0.4 & 0.2 \\ 0.6 & 0.7 & 0.5 \\ 0.9 & 0.8 & 0.7 \\ 1.0 & 1.2 & 1.3 \\ 1.4 & 1.6 & 1.5 \\ 1.8 & 1.7 & 1.9 \end{bmatrix}$$

q1	.4	.1	.8
q2	.5	.8	.6
q3	.5	.8	.6
...			
q6	.0	.8	.5

k1	.4	.1	.8
k2	.5	.8	.6
k3	.5	.8	.6
...			
k6	.0	.8	.5

v1	.4	.1	.8
v2	.5	.8	.6
v3	.5	.8	.6
...			
v6	.0	.8	.5

What is the purpose of q, k & v vectors?



Values (v):

How This Relates to Transformers

Alice's Query (q):	"Who can help me with Physics?"
Alice compares her query with everyone's keys (k):	Alice's own key: "I know Algebra." → Not a match. Bob's key: "I know Physics." → Match! Charlie's key: "I know Poetry." → Not a match.
Based on the match, Alice retrieves Bob's value :	Bob's value: "I'll help you understand Newton's Laws."

Summary

1

Query (q): "What I want to know" (Alice, Bob, and Charlie's questions).

2

Key (k): "What I know" (Alice, Bob, and Charlie's expertise).

3

Value (v): "What I provide if asked" (the actual help or information).

Computing attention score

Created for training and reference purposes

Example

Sentiment Analysis
On IMDB data

Input: A Movie Review

Imagine a review like

"The movie had a slow start, but the acting was phenomenal, and the ending was brilliant."

tokenized into a sequence of tokens (e.g., subwords or words)

[The, movie, had, a, slow, start, but, the, acting, was, phenomenal, and, the, ending, was, brilliant]

Initial Embeddings

Embedding

- Each token is mapped to a **vector** using pre-trained embeddings (e.g., from BERT or GPT).

Example

- "The" → [0.1, 0.5, 0.2]
- "movie" → [0.6, 0.8, 0.3]
- "phenomenal" → [1.2, 0.9, 0.7]
...and so on for all tokens.

Project Tokens to q, k and v Vectors

Each token's embedding is projected into three separate spaces using **learned weight matrices**

Query (q): What each token "wants to know" (its intent in the context).

Key (k): What each token "represents" (its signal or identity).

Value (v): What information each token "carries" (its content).



For each token, these vectors are computed as:

$$Q = \text{Embedding} \cdot W_Q$$

$$K = \text{Embedding} \cdot W_K$$

$$V = \text{Embedding} \cdot W_V$$

Attention Computation (Focus on Relevant Tokens)

- **Example: Focus on "Phenomenal"**
- When analyzing the review for sentiment, the token "phenomenal" might be important.
- The **query (q)** of "phenomenal" is compared to the **keys (k)** of all tokens in the sequence to compute relevance scores (dot products).

Example:

- Compare $q_{\text{"phenomenal"}}$ to $k_{\text{"slow"}}$, $k_{\text{"acting"}}$, $k_{\text{"phenomenal"}}$ etc.
- Higher scores mean the token is more relevant to "phenomenal."

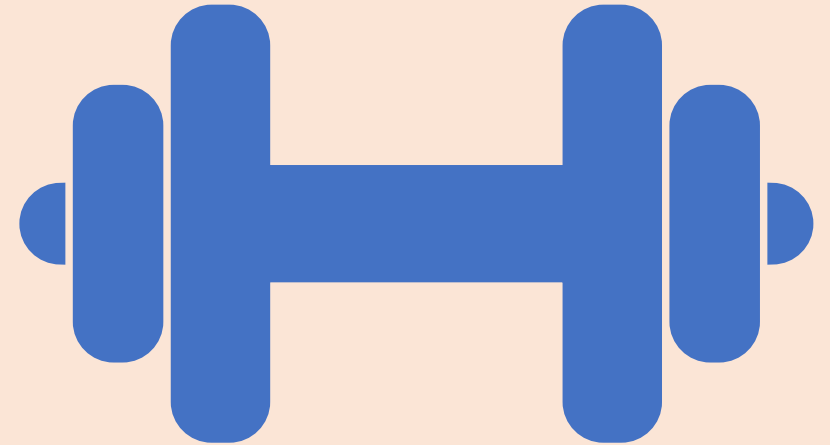
Attention weights

- The dot product scores are normalized using softmax to create attention weights:
 - Attention Weight = $\text{softmax}(q \cdot k)$
- Example:
 - "Phenomenal" attends highly to "acting" and "brilliant."
 - Low attention to "slow" or "start."



Use Attention to Aggregate Information

- The attention weights are used to weight the **values (v)** of all tokens.
- For "phenomenal," it might aggregate:
 - High weight on "acting" (value: [0.8, 0.9, 0.7])
 - Medium weight on "brilliant" (value: [1.0, 0.8, 0.9])
 - Low weight on "slow" (value: [0.2, 0.1, 0.3])
- Result: A context-aware representation of "phenomenal" incorporating its relationship with other tokens.



Final Layer: Classification

01

Each token's final representation is **aggregated** (e.g., using [CLS] token or pooling).

02

The aggregated vector **represents** the review as a whole, capturing relationships and context.

03

This vector is fed to a **classification head** (a simple feedforward layer) to predict sentiment (e.g., positive or negative).

Why Attention Helps in Sentiment Analysis

Contextual Understanding

Words like "slow" might be negative in one context but irrelevant when followed by "phenomenal acting."

Attention lets "phenomenal" focus on "acting" while ignoring "slow."

Long-Term Dependencies

Attention allows the model to connect distant words, like "acting" and "phenomenal," across long sequences.

Dynamic Weighting

Instead of treating all tokens equally, attention highlights important tokens (e.g., "brilliant," "phenomenal") for sentiment.

Example

A simple QnA application

Scenario

Passage

- *"Alice and Bob went to the park. Alice brought a basket and Bob carried a kite. They had a great time playing with the kite."*

Question

- *"Who carried the kite?"*

Tokenization and Embedding

Passage tokens

["Alice", "and", "Bob", "went", "to", "the", "park", ".", "Alice", "brought", "a", "basket", ",", "and", "Bob", "carried", "a", "kite", ".", "They", "had", "a", "great", "time", "playing", "with", "the", "kite", "."]

Question tokens

["Who", "carried", "the", "kite", "?"]

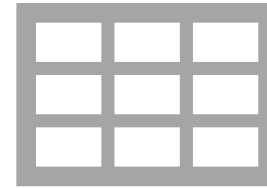
Each token is converted into an embedding (e.g., using pre-trained embeddings like BERT).

Encoding Context with Attention



transformer processes the concatenated sequence of **passage + question**

Input: [CLS] Passage tokens [SEP]
Question tokens [SEP].



Each token generates its query (q), key (k), and value (v) vectors using learned weight matrices.

Attention Scores (Focus Calculation)

- The model calculates **attention scores** using $q \cdot k$:
- **For the token "carried" in the question:**
 - The query vector of "carried" is compared with all key vectors from the passage tokens.
 - The dot product $q \cdot k$ measures similarity to find relevant tokens in the passage.
- **Intuition:**
 - The query for "carried" looks for related information in the passage.
 - High similarity scores are expected for tokens like "Bob" and "carried" in the passage.

Softmax for Attention Weights

- The raw attention scores are passed through a softmax function to normalize them into probabilities.
- For example:
 - **Token: "Bob"** gets 0.7 (high attention weight).
 - **Token: "carried"** gets 0.2 (medium attention weight).
- Other tokens like "park" or "basket" get near-zero weights.

Thanks !!

